

한충희 — 프로젝트 포트폴리오

클라이언트 프로그래머(신입) · Unity · C#

세 번의 프로젝트를 거치며 개인 → 팀으로, 기능 구현 → 시스템 설계로 발전했습니다.

프로젝트 한눈에 (성장 과정)

기간	프로젝트	형태	장르	내 역할	AI 활용
2026.03	Scene Thriller	개인	1인칭 사운드 퍼즐 공포 방탈출	기획·구현 전반	웹 챗봇
2026.04~05	LuckyRandomDefense	팀	2D 랜덤 디펜스	팀장·PM·백엔드·레벨	웹 챗봇
2026.06~	우드득(Wooduduk)	팀	실시간 멀티 스코어어택	클라이언트 시스템	AI 에이전트

1. Scene Thriller — 1인칭 사운드 퍼즐 공포 방탈출 (개인 프로젝트)

- 개발 기간: 2026.03.03 ~ 03.23 (약 3주)
- 개요: 소리를 단서로 방을 탈출하는 1인칭 사운드 퍼즐 공포 게임. 혼자 기획하고 구현한 첫 프로젝트.
- 내 역할: 기획부터 구현까지 전반.
- 한 일
 - 필요한 리소스(텍스처·사운드 등)를 직접 구해 방 환경을 구성.
 - 방 텍스처와 상호작용·퍼즐 등 핵심 게임 기능을 직접 구현.
- 배운 점: 리소스 수급부터 기능 구현까지, 게임 하나를 끝까지 완성해 본 경험.
- AI 활용: 웹 챗봇 AI를 코드·문제 해결의 보조 도구로 활용.

2. LuckyRandomDefense — 2D 랜덤 디펜스 (팀 프로젝트)

- 개발 기간: 2026.04.17 ~ 05.26 (약 6주)
- 개요: 랜덤으로 유닛을 배치해 밀려오는 웨이브를 방어하는 2D 랜덤 디펜스 게임. 첫 팀 프로젝트.
- 내 역할: 팀장 · PM · 데이터 백엔드(PlayFab) · 레벨 디자인.
- 한 일

- **PlayFab 클라우드 데이터 백엔드** — 게임의 서버 데이터 파트를 맡았습니다.
- **익명 로그인**: 기기 고유 ID(LoginWithCustomID)로 신규 기기는 자동 계정 생성.
- **클라우드 세이브**: 유저 세이브 데이터를 PlayFab Player Data에 JSON으로 저장·복원(비동기 콜백 처리).
- **밸런스 데이터 로딩**: PlayFab Title Data에서 증강·퀘스트 데이터를 받아 런타임 데이터로 변환→각 매니저에 전달하고, 로드 완료 이벤트로 게임 시작을 트리거. 밸런스를 클라이언트 빌드 없이 서버에서 조정할 수 있게 함.
- **팀장·PM**: 일정·역할 분담·작업 분해(WBS)를 관리하고 팀 진행을 조율.
- **레벨 디자인**: 웨이브 구성·난이도 등 스테이지 밸런스 설계.
- **배운 점**: 팀을 이끌며 "서버 데이터가 로드된 뒤에 게임을 시작한다"처럼 데이터 의존 흐름을 안정적으로 통제하는 법을 익힘. 이 PlayFab 경험이 이후 우드득의 Firebase 실시간 데이터로 이어짐.
- **AI 활용**: 웹 챗봇 AI를 개발 보조로 활용.

3. 우드득(Wooduduk) — 실시간 멀티 스코어어택 (팀 프로젝트)

- **개발 기간**: 2026.06.03 ~ 진행 중
- **팀 구성**: 5명 (QA 전담 인원 없이 팀 자체 QA)
- **개요**: 슬롯을 굴려 장작을 패고, 최대 4인이 실시간으로 점수·생존을 겨루는 캐주얼 대전 게임.
- **내 역할**: 클라이언트 시스템 파트 — 확장형 카드 시스템, 자동 검증 체계, 실시간 멀티플레이 세션, 티어·LP 정산, 슬롯 엔진 안정화.
- **핵심 구현(요약)**
 - **확장형 카드 효과 시스템** — 슬롯 코어에 효과 효과 집계 전략(곱·합·최솟값·순차)을 먼저 정의하고, 그 위에 카드 10종을 추가했습니다. **기존 판정 엔진과 기존 카드 8종은 한 줄도 수정하지 않았고 신규 파일도 만들지 않았습니다.** 심볼을 바꾸는 효과처럼 서로 충돌할 수 있는 카드는 원본 스냅샷만 읽도록 설계해, 적용 순서와 무관하게 같은 결과가 나오도록 했습니다.
 - **자동 검증 체계 구축** — QA 전담 인원이 없어 수동 확인만으로는 회귀를 놓친다고 판단했습니다. 테스트 프레임워크가 없는 프로젝트라 에디터 메뉴로 실행하는 검증 도구를 직접 만들어 **카드 효과 48종 케이스를 자동 검증**했습니다. 이 과정에서 카드를 중복 구매하면 효과가 지수적으로 증폭되는 밸런스 결함을 발견해 팀에 리포트했습니다.
 - **전용 서버 없이 Firebase만으로 최대 4인 실시간 대전** — 서버시간 기준 전원 동시 시작, 호스트 마이그레이션.
 - **매칭 시스템** — 티어·판수 기반 + 조건 완화 풀백, 백엔드 미완성 구간을 인터페이스로 격리해 독립 개발.
 - **튜토리얼 UX 개편** — "무엇을 해야 할지 모르겠다"는 피드백을 받아, 상시 노출되는 안내 UI 대신 **필요한 순간에만 대상을 강조하고 행동하면 사라지는** 방식으로 재설계했습니다. 다음 행동을 정하는 판정부를 싹과 무관한 순수 함수로 분리해 자동 검증이 가능하게 했습니다.
 - **기획자용 데이터 파이프라인, AI 빌드 전 코드 점검.**

- **트러블슈팅 4건**(호스트 선출 / 조기 체온 드레인 / 정산 버그 / 무한루프 방어)을 근본 원인까지 추적해 해결.
- ➡ **구현 원리·기술 채택 사유·트러블슈팅 상세**는 별도 기술문서([한충희_우드득_기술문서](#))와 아키텍처 대시보드에서 확인할 수 있습니다.
- **AI 활용: AI 코딩 에이전트 도입** — 빌드 전 코드베이스 전체를 점검해 컨벤션 위반·회귀를 사전 차단.

인게임 화면



게임 HUD — 체온·슬롯·콤보·카드



설원의 모닥불



실시간 멀티 세션 — 본인 담당 파트



슬롯머신

AI 활용의 진화

프로젝트를 거치며 AI를 쓰는 방식이 달라졌습니다.

- 초기 (Scene Thriller · LuckyRandomDefense): 웹 챗봇 AI를 코드·문제 해결의 **보조 도구**로 활용.
- 우드득: **AI 코딩 에이전트**를 도입해, 단순 질의를 넘어 **설계 검토 → 구현 → 검증**으로 이어지는 워크플로우로 확장. 요구사항을 먼저 정리하고 설계를 확정된 뒤 구현에 들어가는 순서를 지켰고, 결과는 반드시 자동 검증과 실제 플레이로 확인했습니다.

AI를 쓸수록 검증이 더 중요해진다는 것을 실제로 겪었습니다. 카드 밸런스를 확인하던 중 AI가 알려준 수치가 실제 게임 데이터와 달랐던 적이 있는데, 코드의 기본값과 시트의 실제 값이 갈라져 있던 것이 원인이었습니다. 이후로는 **수치는 반드시 실제 데이터에서 확인하는 절차**를 두었습니다.

AI가 개발의 일상이 된 지금, 저는 'AI와 경쟁'이 아니라 '**AI의 결과를 검증하고 다듬어 팀에 보탬이 되는**' 방향으로 제 역할을 잡아 가고 있습니다.